Отчет по практической работе № 7

Основы баз данных и SQL

Образовательная организация: _____

Институт перспективных технологий и индустриального программирования

Дисциплина: Технологии разработки приложений на базе фреймворков

Группа: ЭФБО-12-24. ФИО: _____

Вариант: Система учета кухонного инвентаря. Дата: 22.10.2026

1. Цель работы

Спроектировать реляционное хранилище системы учета кухонного инвентаря, перенести данные из JSON в SQLite, выполнить SQL-запросы через sqlite3 и показать результаты на страницах Django без использования ORM.

2. Материалы Яндекс Практикума

Заданием предусмотрены материалы раздела 05 «Базы данных и Django ORM»: реляционные базы, SQL и первые запросы, работа из Python, получение и фильтрация данных, сортировка, LIMIT/OFFSET, агрегаты и группировка. Прохождение личного кабинета не подтверждено. ORM, Django-модели и миграции в ПР7 не используются.

3. База данных и перенос данных

Создан файл data/inventory.sqlite. В sql/schema.sql описаны таблицы categories, conditions, rooms, items, movements. Во всех таблицах id INTEGER является первичным ключом. Названия и время хранятся в TEXT, количества, цены в копейках и логические признаки - в INTEGER. Поле rooms.description допускает NULL.

items ссылается на категорию, состояние и текущее помещение; movements - на предмет, исходное и целевое помещения. Внешние ключи включаются через PRAGMA foreign_keys = ON. Ограничения CHECK запрещают неположительные количества и цены, неверный статус и перемещение внутри одного помещения.

Модуль db/database.py выполняет sqlite3.connect() и задает row_factory = sqlite3.Row. Скрипт db/import_json.py проверяет JSON, переносит справочники и записи через executemany(), сохраняет ID и связи, вызывает commit(). Ошибка вставки приводит к rollback. Замена заполненной базы требует явного --replace.

4. SQL и страницы проекта

В sql/queries.sql сохранены 16 именованных запросов с комментариями. Они загружаются приложением из этого же файла. Включены списки помещений, категорий и состояний; WHERE для каталога и исправных предметов; JOIN для карточек и журнала; COUNT/SUM/MIN/MAX/AVG; три группировки GROUP BY; HAVING; сортировка и LIMIT/OFFSET. Минимум два запроса каждого обязательного вида обеспечен.

Все 16 запросов используются веб-приложением. Пользовательские значения передаются отдельно в параметры ?, а не подставляются в SQL. Представления получают sqlite3.Row, обращаются к полям по именам и передают данные в контекст шаблонов.

Страницы	Результат SQL
/items/, /items/available/	Поиск и исправные предметы по помещению
/items/<id>/, /movements/<id>/	Связанные сведения через JOIN
/rooms/, /movements/	Помещения, журнал по дате/статусу
/, /statistics/	Сводка, цены, GROUP BY и HAVING

Пример бронирований из методички адаптирован к своей теме: реализован поиск пригодного инвентаря по помещению и просмотр перемещений по дате. Пустые результаты показываются без ошибки. Некорректные параметры дают 400, неизвестный ID - 404, недоступная SQLite - 503. JSON и консоль ПР3 сохранены как предыдущая реализация; сайт читает только SQLite. README обновлен.

5. Проверка и результат

В базе: 5 категорий, 3 состояния, 4 помещения, 8 партий и 3 перемещения (2 выполнены, 1 отменено). Количество предметов - 8, исправных - 6, стоимость - 6 550.50 руб. PRAGMA integrity_check вернул ok, foreign_key_check не обнаружил нарушений.

python manage.py check - без замечаний; pytest - 104 теста прошли; flake8 . --extend-exclude=venv - без замечаний. Все 16 запросов выполнены через Python API и CLI (python -m sqlite3). Проверены импорт, откат, внешние ключи, поиск, группировки, пустая база, неверные параметры и чтение SQLite независимо от JSON. Сервер и HTTP-ответы страниц и static проверены.

6. Вывод

Создан самостоятельный SQL-слой и обеспечен полный путь SQLite → SQL → sqlite3 → Django views → context → шаблон. Структура и интерфейс подготовлены для последующего перехода к Django ORM.